Informe de Implementacion de Funcionalidades

Proyecto Safari Kids — Sistema de Peluqueria Infantil

1. Introduccion

El presente informe documenta el conjunto de funcionalidades incorporadas al sistema Safari Kids, orientadas a mejorar la gestion de clientes, la atencion presencial y las reservas, asi como la comunicacion automatica con el cliente por correo. Cada bloque de codigo indica, en su barra superior, el archivo del proyecto al que pertenece y la mejora a la que corresponde.

Funcionalidades implementadas:

- A. Registro de clientes con DNI y reclamacion de cuentas provisionales.
- B. Reservas presenciales (walk-in) registradas por el personal.
- C. Seleccion de estilista al momento de reservar.
- D. Cambio de contrasena con confirmacion por correo.
- E. Notificaciones por correo electronico (Gmail SMTP).

2. Tecnologias utilizadas

- Backend: Java 17, Spring Boot 3.4.1 (Web, Data JPA, Security, Mail), JWT.
- Persistencia: JPA/Hibernate sobre MySQL (local) y PostgreSQL (produccion).
- Correo: spring-boot-starter-mail (JavaMailSender) sobre Gmail SMTP.
- Frontend: React (Vite), React Query, react-hot-toast, Axios.

3. Funcionalidades implementadas

3.A. Registro con DNI y reclamacion de cuentas provisionales

El registro exige un DNI valido (8 digitos) como identificador unico. Si un cliente fue atendido de forma presencial (walk-in) y tiene una cuenta provisional asociada a su DNI, al registrarse por la web con el mismo DNI el sistema reclama (vincula) esa cuenta en lugar de duplicarla, conservando su historial de reservas.

backend/.../model/Cliente.java — 3.A Registro con DNI

```java
@Column(length = 8, unique = true)
private String dni;

/** false = cuenta provisional creada en tienda (walk-in), aun no
 *  reclamada. null/true = cuenta normal o ya reclamada. */
private Boolean reclamada;
```

backend/.../repository/ClienteRepository.java — 3.A Registro con DNI

```
Optional<Cliente> findByDni(String dni);
```

backend/.../dto/RegisterRequest.java — 3.A Registro con DNI

```
private String dni;
```

backend/.../controller/AuthController.java

3.A Registro con DNI

```java
@Transactional
@PostMapping("/register")
public ResponseEntity<ApiResponse<LoginResponse>> register(@RequestBody RegisterRequest req)
{
    String dni = req.getDni() == null ? null : req.getDni().trim();
    if (dni == null || !dni.matches("\\d{8}"))
        return ResponseEntity.badRequest().body(ApiResponse.error("El DNI debe tener 8
digitos"));

    Rol rolCliente = rolRepo.findByNombre("Cliente")
        .orElseThrow(() -> new RuntimeException("Rol CLIENTE no encontrado"));

    // Reclamacion de cuenta provisional (walk-in) por DNI
    Optional<Cliente> porDni = clienteRepo.findByDni(dni);
    if (porDni.isPresent()) {
        Cliente cliente = porDni.get();
        if (!Boolean.FALSE.equals(cliente.getReclamada()))
            return ResponseEntity.badRequest().body(
                ApiResponse.error("Este DNI ya tiene una cuenta registrada. Inicia
sesion."));

        // Cuenta provisional -> vincular correo/contrasena reales del cliente
        Usuario usuario = cliente.getUsuario();
        if (!usuario.getCorreo().equals(req.getCorreo()) &&
usuarioRepo.existsByCorreo(req.getCorreo()))
            return ResponseEntity.badRequest().body(ApiResponse.error("El correo ya esta
registrado"));

        usuario.setNombres(req.getNombres());
        usuario.setApellidos(req.getApellidos());
        usuario.setCorreo(req.getCorreo());
        usuario.setContrasena(passwordEncoder.encode(req.getContrasena()));
        usuarioRepo.save(usuario);

        cliente.setReclamada(true);
        clienteRepo.save(cliente);

        UserDetails ud = userDetailsService.loadUserByUsername(usuario.getCorreo());
        String token = jwtUtil.generateToken(ud);
        LoginResponse resp = new LoginResponse(token, usuario.getCorreo(),
            usuario.getNombres() + " " + usuario.getApellidos(), rolCliente.getNombre());
        emailService.enviarBienvenida(usuario.getCorreo(), usuario.getNombres());
        return ResponseEntity.ok(ApiResponse.ok(
            "Cuenta vinculada! Recuperamos tu historial de reservas", resp));
    }

    // Registro nuevo
    if (usuarioRepo.existsByCorreo(req.getCorreo()))
        return ResponseEntity.badRequest().body(ApiResponse.error("El correo ya esta
registrado"));

    Usuario usuario = Usuario.builder()
        .nombres(req.getNombres()).apellidos(req.getApellidos())
        .rol(rolCliente).build();
    usuarioRepo.save(usuario);
```

```
    Cliente cliente = Cliente.builder()
        .usuario(usuario).dni(dni).reclamada(true)
        .telefono(req.getTelefono()).build();
    clienteRepo.save(cliente);

    emailService.enviarBienvenida(usuario.getCorreo(), usuario.getNombres());
    return ResponseEntity.ok(ApiResponse.ok("Registro exitoso", resp));
}
```

3.B. Reservas presenciales (walk-in) por el personal

El personal (ADMIN o EMPLEADO) registra una reserva de un cliente que llega al salon. Ingresa el DNI: si existe, se usa esa cuenta; si no, se crea una cuenta provisional con correo ficticio (dni@walkin.safarikids) y contrasena aleatoria, marcada reclamada=false, para que el cliente la reclame luego desde la web.

backend/.../config/SecurityConfig.java — 3.B Reservas presenciales

```
.requestMatchers(HttpMethod.POST, "/api/reservas/presencial")
    .hasAnyRole("ADMIN", "EMPLEADO")
```

backend/.../controller/ReservaController.java — 3.B Reservas presenciales

```
@Transactional
@PostMapping("/presencial")
public ResponseEntity<ApiResponse<Map<String, Object>>> crearPresencial(
        @RequestBody PresencialReservaRequest req) {
    String dni = req.getDni() == null ? null : req.getDni().trim();
    if (dni == null || !dni.matches("\\d{8}"))
        return ResponseEntity.badRequest().body(ApiResponse.error("El DNI debe tener 8
digitos"));

    Cliente cliente = clienteRepo.findByDni(dni).orElseGet(() -> {
        Rol rolCliente = rolRepo.findByNombre("Cliente")
            .orElseThrow(() -> new RuntimeException("Rol Cliente no encontrado"));
        Usuario usuario = Usuario.builder()
            .nombres(req.getNombres() != null ? req.getNombres() : "Cliente")
            .apellidos(req.getApellidos() != null ? req.getApellidos() : dni)
            .correo(dni + "@walkin.safarikids")
            .contrasena(passwordEncoder.encode(UUID.randomUUID().toString()))
            .estado(Usuario.EstadoUsuario.Activo)
            .fechaRegistro(LocalDate.now()).rol(rolCliente).build();
        usuarioRepo.save(usuario);
        return clienteRepo.save(Cliente.builder()
            .usuario(usuario).dni(dni).reclamada(false)
            .telefono(req.getTelefono()).build());
    });

    Map<String, Object> result = registrarReserva(cliente, req.getFecha(),
req.getHoraInicio(),
        req.getIdServicio(), req.getIdAsiento(), req.getIdEmpleado(), req.getLocal(),
req.getObservaciones());
    result.put("dni", dni);
    result.put("cuentaNueva", Boolean.FALSE.equals(cliente.getReclamada()));
    return ResponseEntity.ok(ApiResponse.ok("Reserva presencial creada", result));
}
```

backend/.../controller/ClienteController.java — 3.B Reservas presenciales

```
/** Busca un cliente por DNI (para autocompletar la reserva presencial). */
@Transactional(readOnly = true)
@GetMapping("/dni/{dni}")
public ResponseEntity<ApiResponse<Cliente>> getByDni(@PathVariable String dni) {
    return repository.findByDni(dni).map(c -> ResponseEntity.ok(ApiResponse.ok(c)))
        .orElse(ResponseEntity.status(404).body(ApiResponse.error("Cliente no
encontrado")));
}
```

frontend/src/api/clientes.js y reservas.js — 3.B Reservas presenciales

```
// clientes.js
porDni: (dni) => api.get(BASE + '/dni/' + dni),

// reservas.js
crearPresencial: (data) => api.post(BASE + '/presencial', data),
```

Frontend: se agrego el componente ReservaPresencialModal.jsx, que busca por DNI, autocompleta datos si el cliente existe y registra la reserva presencial.

3.C. Seleccion de estilista al reservar

Al reservar, el cliente puede elegir un estilista de la sede o dejar 'Cualquiera disponible'. Si elige uno, se respeta; si no, el sistema asigna al estilista con menor carga del dia (asignacion balanceada). La logica de creacion de reserva (web y presencial) se unifico en el metodo registrarReserva.

backend/.../controller/EmpleadoController.java — 3.C Seleccion de estilista

```java
/** Estilistas activos de una sede, para que el cliente elija al reservar. */
@Transactional(readOnly = true)
@GetMapping("/disponibles")
public ResponseEntity<ApiResponse<List<Empleado>>> disponibles(
        @RequestParam(required = false) String sucursal) {
    List<Empleado> activos = (sucursal != null && !sucursal.isBlank())
        ? repository.findBySucursalAndEstado(sucursal, Empleado.EstadoEmpleado.Activo)
        : repository.findByEstado(Empleado.EstadoEmpleado.Activo);
    return ResponseEntity.ok(ApiResponse.ok(activos));
}
```

backend/.../controller/ReservaController.java — 3.C Seleccion de estilista

```java
/** Si el cliente eligio un estilista, lo usa; si no, asigna por carga de trabajo del dia y sede. */
private Empleado asignarEmpleado(LocalDate fechaDate, String local, Integer idEmpleado) {
    if (idEmpleado != null) {
        Empleado empleado = empleadoRepo.findById(idEmpleado)
            .orElseThrow(() -> new RuntimeException("Estilista no encontrado"));
        if (empleado.getEstado() != Empleado.EstadoEmpleado.Activo)
            throw new RuntimeException("El estilista seleccionado no esta disponible");
        return empleado;
    }

    List<Empleado> activos;
    if (local != null && !local.isBlank()) {
        activos = empleadoRepo.findBySucursalAndEstado(local,
Empleado.EstadoEmpleado.Activo);
        if (activos.isEmpty()) activos =
empleadoRepo.findByEstado(Empleado.EstadoEmpleado.Activo);
    } else {
        activos = empleadoRepo.findByEstado(Empleado.EstadoEmpleado.Activo);
    }
    if (activos.isEmpty()) throw new RuntimeException("Sin empleados activos");

    Map<Integer, Long> carga = repository.findByFechaReserva(fechaDate).stream()
        .filter(r -> r.getEmpleado() != null && r.getEstado() !=
Reserva.EstadoReserva.Cancelado)
        .collect(Collectors.groupingBy(r -> r.getEmpleado().getIdEmpleado(),
Collectors.counting()));

    long minCarga = activos.stream()
        .mapToLong(e -> carga.getOrDefault(e.getIdEmpleado(), 0L)).min().orElse(0);

    List<Empleado> candidatos = activos.stream()
        .filter(e -> carga.getOrDefault(e.getIdEmpleado(), 0L) == minCarga)
        .collect(Collectors.toList());

    return candidatos.get(new Random().nextInt(candidatos.size()));
}
```

frontend/src/api/empleados.js — 3.C Seleccion de estilista

```javascript
disponibles: (sucursal) => api.get(BASE + '/disponibles', { params: { sucursal } }),
```

3.D. Cambio de contrasena con confirmacion

El cliente cambia su contrasena desde su cuenta: se valida la actual, se guarda la nueva cifrada y se envia un correo de confirmacion de seguridad.

backend/.../controller/UsuarioController.java — 3.D Cambio de contrasena

```
u.setContrasena(passwordEncoder.encode(req.getNueva()));
repository.save(u);
emailService.enviarCambioPassword(u.getCorreo(), u.getNombres());
return ResponseEntity.ok(ApiResponse.ok("Contrasena actualizada", null));
```

frontend/src/components/CambiarPasswordModal.jsx — 3.D Cambio de contrasena

```
const handleSubmit = async (e) => {
  e.preventDefault()
  if (!form.actual || !form.nueva) { toast.error('Completa todos los campos'); return }
  if (form.nueva !== form.confirmar) { toast.error('Las nuevas contrasenas no coinciden');
return }
  // ... llamada a la API para actualizar la contrasena ...
}
```

3.E. Notificaciones por correo electronico (Gmail SMTP)

Modulo de correos transaccionales (bienvenida, confirmacion/cancelacion de reserva, cambio de contrasena) via Gmail SMTP con JavaMailSender. Envio asincrono y tolerante a fallos: nunca bloquea la operacion principal. Sin credenciales, el envio se omite. Las credenciales se leen de variables de entorno, no del codigo.

backend/pom.xml — 3.E Correo (Gmail SMTP)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

backend/.../resources/application.properties — 3.E Correo (Gmail SMTP)

```
spring.mail.host=${MAIL_HOST:smtp.gmail.com}
spring.mail.port=${MAIL_PORT:587}
spring.mail.username=${MAIL_USERNAME:}
spring.mail.password=${MAIL_PASSWORD:}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
app.mail.from-name=${MAIL_FROM_NAME:Safari Kids}
```

backend/.../service/EmailService.java

3.E Correo (Gmail SMTP)

```java
package com.safarikids.service;

import jakarta.mail.internet.InternetAddress;
import jakarta.mail.internet.MimeMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;

import java.nio.charset.StandardCharsets;
import java.util.concurrent.CompletableFuture;

/**
 * Envío de correos transaccionales vía Gmail SMTP (JavaMailSender).
 * Se configura con las variables de entorno MAIL_USERNAME y MAIL_PASSWORD
 * (usar una "contraseña de aplicación" de Gmail, no la contraseña normal).
 * Si no hay credenciales configuradas, el envío se omite silenciosamente (la app sigue
funcionando).
 * Los envíos son "fire-and-forget": nunca bloquean ni rompen la operación principal.
 */
@Service
public class EmailService {

    private static final Logger log = LoggerFactory.getLogger(EmailService.class);

    private final JavaMailSender mailSender;

    @Value("${spring.mail.username:}")
    private String username;

    @Value("${app.mail.from-name:Safari Kids}")
    private String fromName;

    @Value("${app.frontend-url:http://localhost:5173}")
    private String frontendUrl;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    private boolean habilitado() {
        return username != null && !username.isBlank();
    }

    /** ¿Es un correo real al que podemos escribir? (las cuentas walk-in usan un correo
ficticio) */
    private boolean correoValido(String correo) {
        return correo != null && correo.contains("@") &&
!correo.endsWith("@walkin.safarikids");
    }

    /** Envío genérico, asíncrono y tolerante a fallos. */
    public void enviar(String destino, String asunto, String html) {
        if (!habilitado()) {
```

```
            log.info("[Email] MAIL_USERNAME no configurado — se omite envío a {}", destino);
            return;
        }
        if (!correoValido(destino)) {
            log.info("[Email] Destino no válido o cuenta provisional, se omite: {}",
destino);
            return;
        }

        CompletableFuture
            .runAsync(() -> {
                try {
                    MimeMessage msg = mailSender.createMimeMessage();
                    MimeMessageHelper helper = new MimeMessageHelper(msg, false,
StandardCharsets.UTF_8.name());
                    // Gmail obliga a que el remitente sea la cuenta autenticada; solo
personalizamos el nombre.
                    helper.setFrom(new InternetAddress(username, fromName,
StandardCharsets.UTF_8.name()));
                    helper.setTo(destino);
                    helper.setSubject(asunto);
                    helper.setText(html, true);
                    mailSender.send(msg);
                    log.info("[Email] Enviado a {} — {}", destino, asunto);
                } catch (Exception e) {
                    log.warn("[Email] Error enviando a {}: {}", destino, e.getMessage());
                }
            });
    }

    //
    // Correos específicos
    //

    public void enviarBienvenida(String correo, String nombre) {
        String html = layout("¡Bienvenido a Safari Kids! ",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + "<p>¡Gracias por unirte a la familia <b>Safari Kids</b>! Ya puedes reservar
citas, "
            + "ver tu historial y comprar productos para tus peques.</p>"
            + "<p style=\"text-align:center\"><a href=\"" + frontendUrl + "/mi-cuenta\" "
            + "style=\"display:inline-block;background:#1B7A3E;color:#fff;text-
decoration:none;"
            + "padding:12px 28px;border-radius:999px;font-weight:bold\">Ir a mi cuenta</a>
</p>");
        enviar(correo, "¡Bienvenido a Safari Kids! ", html);
    }

    public void enviarConfirmacionReserva(String correo, String nombre, String codigo,
            String servicio, String fecha, String hora, String local, boolean efectivo) {
        String estado = efectivo
            ? "<p>Tu reserva quedó <b>registrada</b>. Presenta tu código al llegar para
pagar en el salón.</p>"
            : "<p>¡Tu reserva está <b>confirmada</b> y pagada! Te esperamos. </p>";
        String html = layout("Reserva " + (efectivo ? "registrada" : "confirmada") + " ",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + estado
            + detalle("Código", codigo)
            + detalle("Servicio", servicio)
```

```
            + detalle("Fecha", fecha)
            + detalle("Hora", hora)
            + detalle("Sede", local));
        enviar(correo, "Tu reserva en Safari Kids — " + codigo, html);
    }

    public void enviarCancelacionReserva(String correo, String nombre, String codigo, String
fecha) {
        String html = layout("Reserva cancelada",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + "<p>Tu reserva <b>" + esc(codigo) + "</b>"
            + (fecha != null ? " del <b>" + esc(fecha) + "</b>" : "")
            + " ha sido <b>cancelada</b>.</p>"
            + "<p>Si no fuiste tú o deseas reagendar, ingresa a tu cuenta cuando quieras.
</p>");
        enviar(correo, "Reserva cancelada — Safari Kids", html);
    }

    public void enviarCambioPassword(String correo, String nombre) {
        String html = layout("Tu contraseña fue cambiada ",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + "<p>Te confirmamos que la contraseña de tu cuenta de <b>Safari Kids</b> se
cambió correctamente.</p>"
            + "<p>Si <b>no</b> realizaste este cambio, contáctanos de inmediato.</p>");
        enviar(correo, "Cambio de contraseña — Safari Kids", html);
    }

    //
    // Helpers de plantilla
    //

    private String detalle(String label, String valor) {
        if (valor == null || valor.isBlank()) return "";
        return "<tr><td style=\"padding:6px 0;color:#888;font-size:14px\">" + esc(label) + "
</td>"
            + "<td style=\"padding:6px 0;text-align:right;font-
weight:bold;color:#1A2E1A;font-size:14px\">"
            + esc(valor) + "</td></tr>";
    }

    private String layout(String titulo, String cuerpo) {
        boolean tieneTabla = cuerpo.contains("<tr>");
        String contenido = cuerpo;
        if (tieneTabla) {
            int idx = cuerpo.indexOf("<tr>");
            contenido = cuerpo.substring(0, idx)
                + "<table style=\"width:100%;border-collapse:collapse;margin:16px 0;"
                + "background:#f9f9f9;border-radius:12px;padding:8px 16px\">"
                + cuerpo.substring(idx) + "</table>";
        }
        return "<div style=\"font-family:Arial,Helvetica,sans-serif;max-width:520px;margin:0
auto;"
            + "background:#fff;border:3px solid #1A2E1A;border-
radius:18px;overflow:hidden\">"
            + "<div style=\"background:linear-
gradient(135deg,#1B7A3E,#33A957);padding:24px;text-align:center\">"
            + "<h1 style=\"color:#fff;margin:0;font-size:22px\">" + esc(titulo) + "</h1>
</div>"
            + "<div style=\"padding:24px;color:#333;font-size:15px;line-height:1.6\">" +
```

```
contenido
            + "<p style=\"margin-top:24px;color:#aaa;font-size:12px;text-align:center\">"
            + "Safari Kids — Peluquería infantil </p></div></div>";
    }

    private String esc(String s) {
        if (s == null) return "";
        return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;");
    }
}
```

4. Pruebas del sistema

Se definieron seis tipos de pruebas, cada una con su respectivo codigo. La barra de cada bloque indica el archivo de prueba correspondiente.

4.1. Pruebas funcionales

Verifican el comportamiento esperado (validacion de DNI, envio de correos, reglas de negocio). JUnit 5 + Mockito.

backend/src/test/.../FuncionalesTest.java

4.1 Pruebas funcionales

```
class FuncionalesTest {

    private JavaMailSender sender;
    private EmailService service;

    @BeforeEach
    void setUp() {
        sender = mock(JavaMailSender.class);
        when(sender.createMimeMessage())
            .thenAnswer(inv -> new JavaMailSenderImpl().createMimeMessage());
        service = new EmailService(sender);
        ReflectionTestUtils.setField(service, "username", "safarikids@gmail.com");
        ReflectionTestUtils.setField(service, "fromName", "Safari Kids");
    }

    @Test
    void correoDeBienvenidaSeEnvia() {
        service.enviarBienvenida("cliente@correo.com", "Ana");
        verify(sender, timeout(2000)).send(any(MimeMessage.class));
    }

    @Test
    void noEnviaACuentaProvisionalWalkin() {
        service.enviar("12345678@walkin.safarikids", "Prueba", "<p>Hola</p>");
        verify(sender, after(500).never()).send(any(MimeMessage.class));
    }

    @Test
    void dniInvalidoNoTieneOchoDigitos() {
        assertThat("1234567".matches("\\d{8}")).isFalse();
        assertThat("12345678".matches("\\d{8}")).isTrue();
    }
}
```

4.2. Pruebas de integracion

Levantan el contexto de Spring Boot y prueban el registro (crea cliente con DNI) y el envio real de correo contra un servidor SMTP de prueba (GreenMail).

backend/src/test/.../IntegracionTest.java

4.2 Pruebas de integracion

```java
@SpringBootTest
@AutoConfigureMockMvc
@TestPropertySource(properties = {
    "spring.mail.host=127.0.0.1", "spring.mail.port=3025",
    "spring.mail.username=safarikids@gmail.com", "spring.mail.password=claveDePrueba"
})
class IntegracionTest {

    @RegisterExtension
    static GreenMailExtension smtp = new GreenMailExtension(ServerSetupTest.SMTP);

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper json;

    @Test
    void registroCreaClienteYEnviaBienvenida() throws Exception {
        String body = json.writeValueAsString(Map.of(
            "nombres", "Ana", "apellidos", "Perez", "dni", "45678912",
            "correo", "ana.prueba@correo.com", "contrasena", "Clave1234!"));

        mockMvc.perform(post("/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON).content(body))
            .andExpect(status().isOk());

        await().atMost(5, SECONDS).until(() -> smtp.getReceivedMessages().length == 1);
        assertThat(smtp.getReceivedMessages()[0].getSubject())
            .contains("Bienvenido a Safari Kids");
    }
}
```

4.3. Pruebas de usabilidad

Se evaluan de forma manual (lista de verificacion) y con una prueba E2E del frontend con Playwright que simula al usuario real.

- El formulario indica que el DNI es obligatorio (8 digitos).
- El mensaje 'Cuenta vinculada' al reclamar una cuenta walk-in es claro.
- La seccion 'Elige a tu estilista' y la opcion 'Cualquiera disponible' se entienden.
- El modal de cambio de contrasena avisa si las contrasenas no coinciden.

frontend/tests/cambio-password.spec.ts — 4.3 Pruebas de usabilidad

```ts
import { test, expect } from '@playwright/test';

test('el cambio de contrasena valida que coincidan', async ({ page }) => {
  await page.goto('http://localhost:5173/mi-cuenta');
  await page.getByRole('button', { name: /cambiar contrasena/i }).click();

  await page.getByLabel(/actual/i).fill('Clave1234!');
  await page.getByLabel(/nueva/i).first().fill('NuevaClave1!');
  await page.getByLabel(/confirmar/i).fill('OtraDistinta9!');
  await page.getByRole('button', { name: /guardar|cambiar/i }).click();

  await expect(page.getByText(/no coinciden/i)).toBeVisible();
});
```

4.4. Pruebas de rendimiento

Comprueban (a) que el envio de correo NO bloquea la respuesta (asincrono) y (b) que el sistema soporta multiples solicitudes concurrentes (k6).

backend/src/test/.../RendimientoTest.java — 4.4 Pruebas de rendimiento

```java
@Test
void enviarNoBloqueaAlUsuario() {
    JavaMailSender sender = mock(JavaMailSender.class);
    when(sender.createMimeMessage())
        .thenAnswer(inv -> new JavaMailSenderImpl().createMimeMessage());
    // Simulamos un servidor SMTP lento (2 segundos)
    doAnswer(inv -> { Thread.sleep(2000); return null; })
        .when(sender).send(any(MimeMessage.class));

    EmailService service = new EmailService(sender);
    ReflectionTestUtils.setField(service, "username", "safarikids@gmail.com");
    ReflectionTestUtils.setField(service, "fromName", "Safari Kids");

    long inicio = System.currentTimeMillis();
    service.enviarBienvenida("cliente@correo.com", "Ana");
    long duracion = System.currentTimeMillis() - inicio;

    // Aunque el SMTP tarde 2s, la llamada debe retornar casi al instante
    assertThat(duracion).isLessThan(500);
}
```

carga.js (ejecucion: k6 run carga.js) 4.4 Pruebas de rendimiento

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 50, duration: '30s',
  thresholds: {
    http_req_duration: ['p(95)<800'], // 95% de respuestas < 800 ms
    http_req_failed:   ['rate<0.01'], // menos de 1% de errores
  },
};

const BASE = 'https://safari-kids-backend.up.railway.app';

export default function () {
  const body = JSON.stringify({ correo: 'demo@correo.com', contrasena: 'Clave1234!' });
  const params = { headers: { 'Content-Type': 'application/json' } };
  const res = http.post(`${BASE}/api/auth/login`, body, params);
  check(res, { 'respuesta recibida': (r) => r.status === 200 || r.status === 401 });
  sleep(1);
}
```

4.5. Pruebas de seguridad

Verifican que (a) /presencial exige rol ADMIN/EMPLEADO, (b) los endpoints protegidos rechazan peticiones sin token JWT y (c) el contenido dinamico de los correos se escapa contra inyeccion de HTML.

backend/src/test/.../SeguridadTest.java — 4.5 Pruebas de seguridad

```java
@SpringBootTest
@AutoConfigureMockMvc
class SeguridadTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private EmailService emailService;

    @Test
    void reservaPresencialRequiereAutenticacion() throws Exception {
        mockMvc.perform(post("/api/reservas/presencial")
                .contentType(MediaType.APPLICATION_JSON).content("{\"dni\":\"12345678\"}"))
            .andExpect(status().is4xxClientError());
    }

    @Test
    @WithMockUser(roles = "CLIENTE")
    void reservaPresencialProhibidaParaClientes() throws Exception {
        mockMvc.perform(post("/api/reservas/presencial")
                .contentType(MediaType.APPLICATION_JSON).content("{\"dni\":\"12345678\"}"))
            .andExpect(status().isForbidden());
    }

    @Test
    void contenidoDinamicoSeEscapaContraInyeccionHtml() {
        String nombreMalicioso = "<script>alert(1)</script>";
        String out = (String) ReflectionTestUtils.invokeMethod(emailService, "esc",
nombreMalicioso);
        assertThat(out).doesNotContain("<script>");
        assertThat(out).contains("&lt;script&gt;");
    }
}
```

4.6. Pruebas de compatibilidad

Verifican que el sistema funciona en los dos motores de base de datos (MySQL local y PostgreSQL produccion) y en los principales navegadores. El contexto debe cargar con H2 en modo PostgreSQL; la misma prueba E2E se ejecuta en varios navegadores con Playwright.

backend/src/test/.../CompatibilidadTest.java — 4.6 Pruebas de compatibilidad

```java
@SpringBootTest
@ActiveProfiles("test") // perfil que usa H2 en memoria (MODE=PostgreSQL)
class CompatibilidadTest {

    @Test
    void elContextoCargaEnMotorCompatible() {
        // Si el contexto carga sin errores, el mapeo JPA es compatible
        // con motores estandar (H2, MySQL, PostgreSQL).
    }
}
```

frontend/playwright.config.ts

4.6 Pruebas de compatibilidad

```ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    { name: 'Chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'Firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'WebKit',   use: { ...devices['Desktop Safari'] } },
    { name: 'Mobile',   use: { ...devices['Pixel 5'] } },
  ],
});
```

5. Conclusiones

- Se implementaron cinco funcionalidades que fortalecen la gestion de clientes y reservas.
- El DNI como identificador unico unifica la atencion presencial y la web sin duplicar cuentas y conservando el historial.
- El modulo de correo, asincrono y tolerante a fallos, garantiza que una falla de envio nunca afecte la operacion principal.
- La seguridad se reforzo con control de acceso por roles, cifrado de contrasenas (BCrypt), credenciales fuera del codigo y SMTP cifrado (STARTTLS).
- Todo se valido con seis tipos de pruebas: funcionales, integracion, usabilidad, rendimiento, seguridad y compatibilidad.